

Dissecting the classic Singleton Pattern implementation



Watch it!

If you're just flipping through the book, don't blindly type in this code; you'll see it has a few issues later in the chapter.

```

public class Singleton {
    private static Singleton uniqueInstance;

    // other useful instance variables here

    private Singleton() {}

    public static Singleton getInstance() {
        if (uniqueInstance == null) {
            uniqueInstance = new Singleton();
        }
        return uniqueInstance;
    }

    // other useful methods here
}

```

Let's rename MyClass to Singleton.

We have a static variable to hold our one instance of the class Singleton.

Our constructor is declared private; only Singleton can instantiate this class!

The getInstance() method gives us a way to instantiate the class and also to return an instance of it.

Of course, Singleton is a normal class; it has other useful instance variables and methods.



Code Up Close

```

if (uniqueInstance == null) {
    uniqueInstance = new Singleton();
}
return uniqueInstance;

```

uniqueInstance holds our *ONE* instance; remember, it is a static variable.

If uniqueInstance is null, then we haven't created the instance yet...

...and, if it doesn't exist, we instantiate Singleton through its private constructor and assign it to uniqueInstance. Note that if we never need the instance, it never gets created; this is lazy instantiation.

If uniqueInstance wasn't null, then it was previously created. We just fall through to the return statement.

By the time we hit this code, we have an instance and we return it.



Patterns Exposed

This week's interview:
Confessions of a Singleton

HeadFirst: Today we are pleased to bring you an interview with a Singleton object. Why don't you begin by telling us a bit about yourself?

Singleton: Well, I'm totally unique; there is just one of me!

HeadFirst: One?

Singleton: Yes, one. I'm based on the Singleton Pattern, which ensures that at any time there is only one instance of me.

HeadFirst: Isn't that sort of a waste? Someone took the time to develop a full-blown class and now all we can get is one object out of it?

Singleton: Not at all! There is power in ONE. Let's say you have an object that contains registry settings. You don't want multiple copies of that object and its values running around—that would lead to chaos. By using an object like me you can ensure that every object in your application is making use of the same global resource.

HeadFirst: Tell us more...

Singleton: Oh, I'm good for all kinds of things. Being single sometimes has its advantages, you know. I'm often used to manage pools of resources, like connection or thread pools.

HeadFirst: Still, only one of your kind? That sounds lonely.

Singleton: Because there's only one of me, I do keep busy, but it would be nice if more developers knew me—many developers run into bugs because they have multiple copies of objects floating around they're not even aware of.

HeadFirst: So, if we may ask, how do you know there is only one of you? Can't anyone with a new operator create a "new you"?

Singleton: Nope! I'm truly unique.

HeadFirst: Well, do developers swear an oath not to instantiate you more than once?

Singleton: Of course not. The truth be told...well, this is getting kind of personal but...I have no public constructor.

HeadFirst: NO PUBLIC CONSTRUCTOR! Oh, sorry, no public constructor?

Singleton: That's right. My constructor is declared private.

HeadFirst: How does that work? How do you EVER get instantiated?

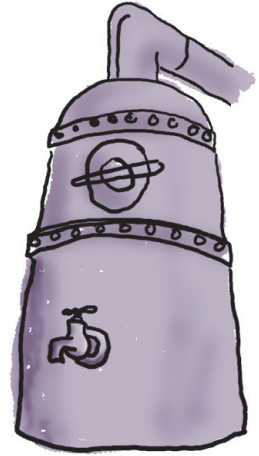
Singleton: You see, to get a hold of a Singleton object, you don't instantiate one, you just ask for an instance. So my class has a static method called `getInstance()`. Call that, and I'll show up at once, ready to work. In fact, I may already be helping other objects when you request me.

HeadFirst: Well, Mr. Singleton, there seems to be a lot under your covers to make all this work. Thanks for revealing yourself and we hope to speak with you again soon!

The Chocolate Factory

Everyone knows that all modern chocolate factories have computer-controlled chocolate boilers. The job of the boiler is to take in chocolate and milk, bring them to a boil, and then pass them on to the next phase of making chocolate bars.

Here's the controller class for Choc-O-Holic, Inc.'s industrial strength Chocolate Boiler. Check out the code; you'll notice they've tried to be very careful to ensure that bad things don't happen, like draining 500 gallons of unboiled mixture, or filling the boiler when it's already full, or boiling an empty boiler!



```
public class ChocolateBoiler {
    private boolean empty;
    private boolean boiled;
```

```
    public ChocolateBoiler() {
        empty = true;
        boiled = false;
    }
```

This code is only started when the boiler is empty!

```
    public void fill() {
        if (isEmpty()) {
            empty = false;
            boiled = false;
            // fill the boiler with a milk/chocolate mixture
        }
    }
```

To fill the boiler it must be empty, and, once it's full, we set the empty and boiled flags.

```
    public void drain() {
        if (!isEmpty() && isBoiled()) {
            // drain the boiled milk and chocolate
            empty = true;
        }
    }
```

To drain the boiler, it must be full (non-empty) and also boiled. Once it is drained, we set empty back to true.

```
    public void boil() {
        if (!isEmpty() && !isBoiled()) {
            // bring the contents to a boil
            boiled = true;
        }
    }
```

To boil the mixture, the boiler has to be full and not already boiled. Once it's boiled, we set the boiled flag to true.

```
    public boolean isEmpty() {
        return empty;
    }
```

```
    public boolean isBoiled() {
        return boiled;
    }
```

```
}
```



Choc-O-Holic has done a decent job of ensuring bad things don't happen, don't you think? Then again, you probably suspect that if two ChocolateBoiler instances get loose, some very bad things can happen.

How might things go wrong if more than one instance of ChocolateBoiler is created in an application?



Sharpen your pencil

Can you help Choc-O-Holic improve their ChocolateBoiler class by turning it into a Singleton?

```
public class ChocolateBoiler {  
    private boolean empty;  
    private boolean boiled;  
  
      
  
     ChocolateBoiler() {  
        empty = true;  
        boiled = false;  
    }  
  
      
  
    public void fill() {  
        if (isEmpty()) {  
            empty = false;  
            boiled = false;  
            // fill the boiler with a milk/chocolate mixture  
        }  
    }  
    // rest of ChocolateBoiler code...  
}
```

Singleton Pattern defined

Now that you've got the classic implementation of Singleton in your head, it's time to sit back, enjoy a bar of chocolate, and check out the finer points of the Singleton Pattern.

Let's start with the concise definition of the pattern:

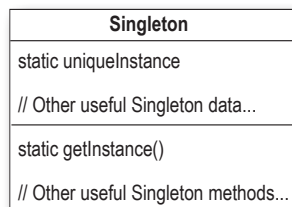
The Singleton Pattern ensures a class has only one instance, and provides a global point of access to it.

No big surprises there. But let's break it down a bit more:

- What's really going on here? We're taking a class and letting it manage a single instance of itself. We're also preventing any other class from creating a new instance on its own. To get an instance, you've got to go through the class itself.
- We're also providing a global access point to the instance: whenever you need an instance, just query the class and it will hand you back the single instance. As you've seen, we can implement this so that the Singleton is created in a lazy manner, which is especially important for resource-intensive objects.

Okay, let's check out the class diagram:

The `getInstance()` method is static, which means it's a class method, so you can conveniently access this method from anywhere in your code using `Singleton.getInstance()`. That's just as easy as accessing a global variable, but we get benefits like lazy instantiation from the Singleton.



The `uniqueInstance` class variable holds our one and only instance of Singleton.

A class implementing the Singleton Pattern is more than a Singleton; it is a general-purpose class with its own set of data and methods.

Hershey, PA ~~Houston~~, we have a problem...

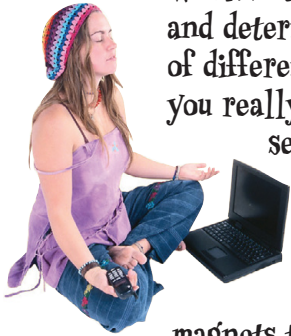
It looks like the Chocolate Boiler has let us down; despite the fact we improved the code using the classic Singleton Pattern, somehow the Chocolate Boiler's `fill()` method was able to start filling the boiler even though a batch of milk and chocolate was already boiling! That's 500 gallons of spilled milk (and chocolate)! What happened!?

We don't know what happened! The new Singleton code was running fine. The only thing we can think of is that we just added some optimizations to the Chocolate Boiler Controller that makes use of multiple threads.



Could the addition of threads have caused this? Isn't it the case that once we've set the `uniqueInstance` variable to the sole instance of `ChocolateBoiler`, all calls to `getInstance()` should return the same instance? Right?

BE the JVM



We have two threads, each executing this code. Your job is to play the JVM and determine whether there is a case in which two threads might get a hold of different boiler objects. Hint:

you really just need to look at the sequence of operations in the `getInstance()` method and the value of `uniqueInstance` to see how they might overlap. Use the code magnets to help you study how the code might interleave to create two boiler objects.

```
ChocolateBoiler boiler =
    ChocolateBoiler.getInstance();
boiler.fill();
boiler.boil();
boiler.drain();
```

```
public static ChocolateBoiler
    getInstance() {
```

```
    if (uniqueInstance == null) {
```

```
        uniqueInstance =
            new ChocolateBoiler();
```

```
    }
```

```
    return uniqueInstance;
```

```
}
```

Make sure you check your answer on page 188 before continuing!

Thread
One

Thread
Two

Value of
`uniqueInstance`

Dealing with multithreading

Our multithreading woes are almost trivially fixed by making `getInstance()` a synchronized method:

```
public class Singleton {
    private static Singleton uniqueInstance;

    // other useful instance variables here

    private Singleton() {}

    public static synchronized Singleton getInstance() {
        if (uniqueInstance == null) {
            uniqueInstance = new Singleton();
        }
        return uniqueInstance;
    }

    // other useful methods here
}
```

By adding the `synchronized` keyword to `getInstance()`, we force every thread to wait its turn before it can enter the method. That is, no two threads may enter the method at the same time.

I agree this fixes the problem. But synchronization is expensive; is this an issue?

Good point, and it's actually a little worse than you make out: the only time synchronization is relevant is the first time through this method. In other words, once we've set the `uniqueInstance` variable to an instance of `Singleton`, we have no further need to synchronize this method. After the first time through, synchronization is totally unneeded overhead!



Can we improve multithreading?

For most Java applications, we obviously need to ensure that the Singleton works in the presence of multiple threads. But it's expensive to synchronize the `getInstance()` method, so what do we do?

Well, we have a few options...

1. Do nothing if the performance of `getInstance()` isn't critical to your application.

That's right; if calling the `getInstance()` method isn't causing substantial overhead for your application, forget about it. Synchronizing `getInstance()` is straightforward and effective. Just keep in mind that synchronizing a method can decrease performance by a factor of 100, so if a high-traffic part of your code begins using `getInstance()`, you may have to reconsider.

2. Move to an eagerly created instance rather than a lazily created one.

If your application always creates and uses an instance of the Singleton, or the overhead of creation and runtime aspects of the Singleton isn't onerous, you may want to create your Singleton eagerly, like this:

```
public class Singleton {  
    private static Singleton uniqueInstance = new Singleton();  
  
    private Singleton() {}  
  
    public static Singleton getInstance() {  
        return uniqueInstance;  
    }  
}
```

Go ahead and create an instance of Singleton in a static initializer. This code is guaranteed to be thread safe!

We've already got an instance, so just return it.

Using this approach, we rely on the JVM to create the unique instance of the Singleton when the class is loaded. The JVM guarantees that the instance will be created before any thread accesses the static `uniqueInstance` variable.

3. Use “double-checked locking” to reduce the use of synchronization in getInstance().

With double-checked locking, we first check to see if an instance is created, and if not, THEN we synchronize. This way, we only synchronize the first time through, just what we want.

Let’s check out the code:

```
public class Singleton {
    private volatile* static Singleton uniqueInstance;

    private Singleton() {}

    public static Singleton getInstance() {
        if (uniqueInstance == null) {
            synchronized (Singleton.class) {
                if (uniqueInstance == null) {
                    uniqueInstance = new Singleton();
                }
            }
        }
        return uniqueInstance;
    }
}
```

Check for an instance and if there isn't one, enter a synchronized block.

Note we only synchronize the first time through!

Once in the block, check again and if it's still null, create an instance.

*The volatile keyword ensures that multiple threads handle the uniqueInstance variable correctly when it is being initialized to the Singleton instance.

If performance is an issue in your use of the getInstance() method, then this method of implementing the Singleton can drastically reduce the overhead.



Watch it!

Double-checked locking doesn't work in Java 1.4 or earlier!

If for some reason you're using an old version of Java, unfortunately, in Java version 1.4 and earlier, many JVMs contain implementations of the volatile keyword that allow improper synchronization for double-checked locking. If you must use a JVM earlier than Java 5, consider other methods of implementing your Singleton.

Meanwhile, back at the Chocolate Factory...

While we've been off diagnosing the multithreading problems, the chocolate boiler has been cleaned up and is ready to go. But first, we have to fix the multithreading problems. We have a few solutions at hand, each with different tradeoffs, so which solution are we going to employ?



Sharpen your pencil

For each solution, describe its applicability to the problem of fixing the Chocolate Boiler code:

Synchronize the `getInstance()` method:

Use eager instantiation:

Double-checked locking:

Congratulations!

At this point, the Chocolate Factory is a happy customer and Choc-O-Holic was glad to have some expertise applied to their boiler code. No matter which multithreading solution you applied, the boiler should be in good shape with no more mishaps. Congratulations—not only have you managed to escape 500 lbs of hot chocolate in this chapter, but you've also been through all the potential problems of the Singleton Pattern.

there are no Dumb Questions

Q: For such a simple pattern consisting of only one class, Singleton sure seems to have some problems.

A: Well, we warned you up front! But don't let the problems discourage you; while implementing Singletons correctly can be tricky, after reading this chapter you're now well informed on the techniques for creating Singletons and should use them wherever you need to control the number of instances you're creating.

Q: Can't I just create a class in which all methods and variables are defined as static? Wouldn't that be the same as a Singleton?

A: Yes, if your class is self-contained and doesn't depend on complex initialization. However, because of the way static initializations are handled in Java, this can get very messy, especially if multiple classes are involved. Often this scenario can result in subtle, hard-to-find bugs involving order of initialization. Unless there is a compelling need to implement your "singleton" this way, it's far better to stay in the object world.

Q: What about class loaders? I heard there's a chance that two class loaders could each end up with their own instance of Singleton.

A: Yes, that is true as each class loader defines a namespace. If you have two or more class loaders, you can load the same class multiple times (once in each class loader). Now, if that class happens to be a Singleton, then since we have more than one version of the class, we also have more than one instance of Singleton. So, if you are using multiple class loaders and Singletons, be careful. One way around this problem is to specify the class loader yourself.

Q: And reflection, and serialization/deserialization?

A: Yes, reflection and serialization/deserialization can also present problems with Singletons. If you're an advanced Java user using reflection, serialization, and deserialization, you'll need to keep that in mind.

Q: Earlier we talked about the loose coupling principle. Isn't a Singleton violating this? After all, every object in our code that depends on the Singleton is going to be tightly coupled to that very specific object.

A: Yes, and in fact this is a common criticism of the Singleton Pattern. The loose coupling principle says to "strive for loosely coupled designs between objects that interact." It's easy for Singletons to violate this principle: if you make a change to the Singleton, you'll likely have to make a change to every object connected to it.

Q: I've always been taught that a class should do one thing and one thing only. For a class to do two things is considered bad OO design. Isn't a Singleton violating this too?

A: You would be referring to the Single Responsibility Principle, and yes, you are correct: the Singleton is responsible not only for managing its one instance (and providing global access), but also for whatever its main role is in your application. So, certainly you could argue it is taking on two responsibilities. Nevertheless, it isn't hard to see that there is utility in a class managing its own instance; it certainly makes the overall design simpler. In addition, many developers are familiar with the Singleton Pattern as it is in wide use. That said, some developers do feel the need to abstract out the Singleton functionality.

Q: I wanted to subclass my Singleton code, but I ran into problems. Is it okay to subclass a Singleton?

A: One problem with subclassing a Singleton is that the constructor is private. You can't extend a class with a private constructor. So, the first thing you'll have to do is change your constructor so that it's public or protected. But then, it's not really a Singleton anymore, because other classes can instantiate it.

If you do change your constructor, there's another issue. The implementation of Singleton is based on a static variable, so if you do a straightforward subclass, all of your derived classes will share the same instance variable. This is probably not what you had in mind. So, for subclassing to work, implementing a registry of sorts is required in the base class.

But what are you really gaining from subclassing a Singleton? Like most patterns, Singleton is not necessarily meant to be a solution that can fit into a library. In addition, the Singleton code is trivial to add to any existing class. Last, if you are using a large number of Singletons in your application, you should take a hard look at your design. Singletons are meant to be used sparingly.

Q: I still don't totally understand why global variables are worse than a Singleton.

A: In Java, global variables are basically static references to objects. There are a couple of disadvantages to using global variables in this manner. We've already mentioned one: the issue of lazy versus eager instantiation. But we need to keep in mind the intent of the pattern: to ensure only one instance of a class exists and to provide global access. A global variable can provide the latter, but not the former. Global variables also tend to encourage developers to pollute the namespace with lots of global references to small objects. Singletons don't encourage this in the same way, but can be abused nonetheless.



I just realized...I think we can solve a lot of the problems with Singleton by using an enum. Is that right?

Ah, good idea!

Many of the problems we've discussed—worrying about synchronization, class loading issues, reflection, and serialization/deserialization issues—can all be solved by using an enum to create your Singleton. Here's how you'd do that:

```
public enum Singleton {
    UNIQUE_INSTANCE;
    // more useful fields here
}

public class SingletonClient {
    public static void main(String[] args) {
        Singleton singleton = Singleton.UNIQUE_INSTANCE;
        // use the singleton here
    }
}
```

And back in the old days, when we had to walk to school, uphill, in the snow, in both directions, Java didn't have enums.

Yep, that's all there is to it. Simplest Singleton ever, right? Now, you might be asking, why did we go through all that earlier with creating a Singleton class with a `getInstance()` method and then synchronizing, and so on? We did that so you really, truly understand how Singleton works. Now that you know, you can go off and use enum whenever you need a Singleton, and still be able to ace that Java interview if the question pops up: "How do you implement a Singleton *without* using enum?"



Can you rework Choc-O-Holic to use an enum? Give it a try.



Tools for your Design Toolbox

You've now added another pattern to your toolbox. Singleton gives you another method of creating objects—in this case, unique objects.

OO Principles

Encapsulate what varies.

Favor composition over inheritance.

Program to interfaces, not implementations.

Strive for loosely coupled designs between objects that interact.

Classes should be open for extension but closed for modification.

Depend on abstractions. Do not depend on concrete classes.

OO Basics

abstraction

encapsulation

polymorphism

inheritance

When you need to ensure you only have one instance of a class running around your application, turn to the Singleton.

OO Patterns

Singleton - Ensure a class only has one instance and provide a global point of access to it.

As you've seen, despite its apparent simplicity, there are a lot of details involved in Singleton's implementation. After reading this chapter, though, you're ready to go out and use Singleton in the wild.



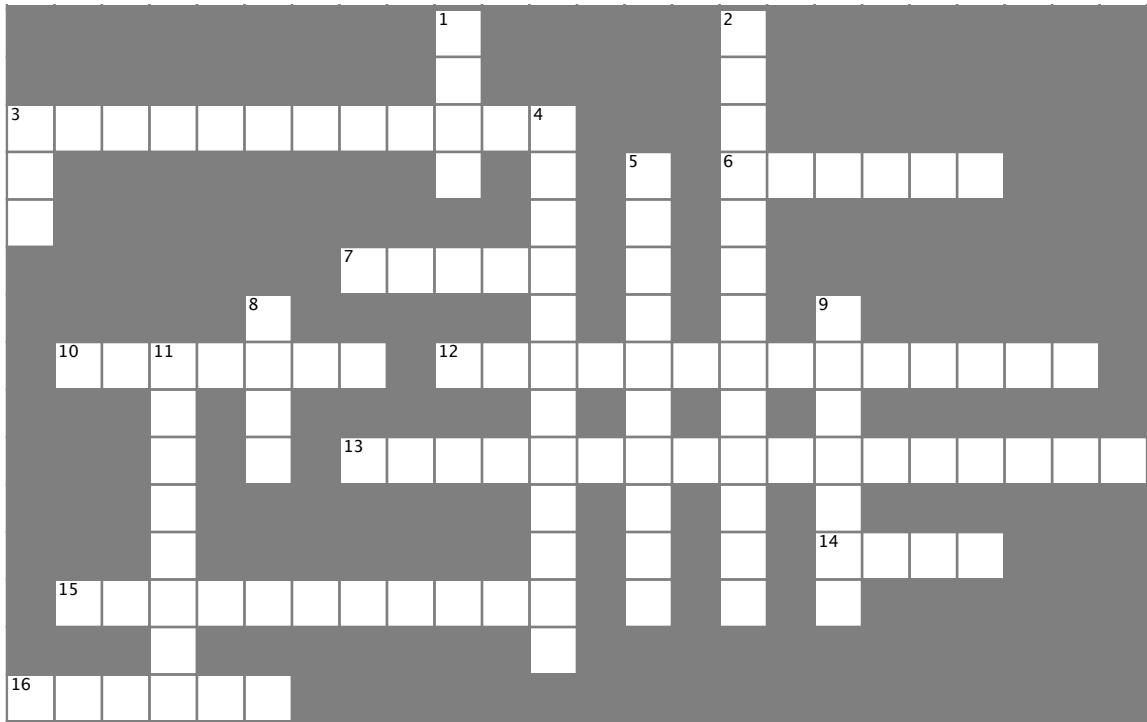
BULLET POINTS

- The Singleton Pattern ensures you have at most one instance of a class in your application.
- The Singleton Pattern also provides a global access point to that instance.
- Java's implementation of the Singleton Pattern makes use of a private constructor, a static method combined with a static variable.
- Examine your performance and resource constraints and carefully choose an appropriate Singleton implementation for multithreaded applications (and we should consider all applications multithreaded!).
- Beware of the double-checked locking implementation; it isn't thread safe in versions before Java 5.
- Be careful if you are using multiple class loaders; this could defeat the Singleton implementation and result in multiple instances.
- You can use Java's enums to simplify your Singleton implementation.



Design Patterns Crossword

Sit back, open that case of chocolate that you were sent for solving the multithreading problem, and have some downtime working on this little crossword puzzle; all of the solution words are from this chapter.

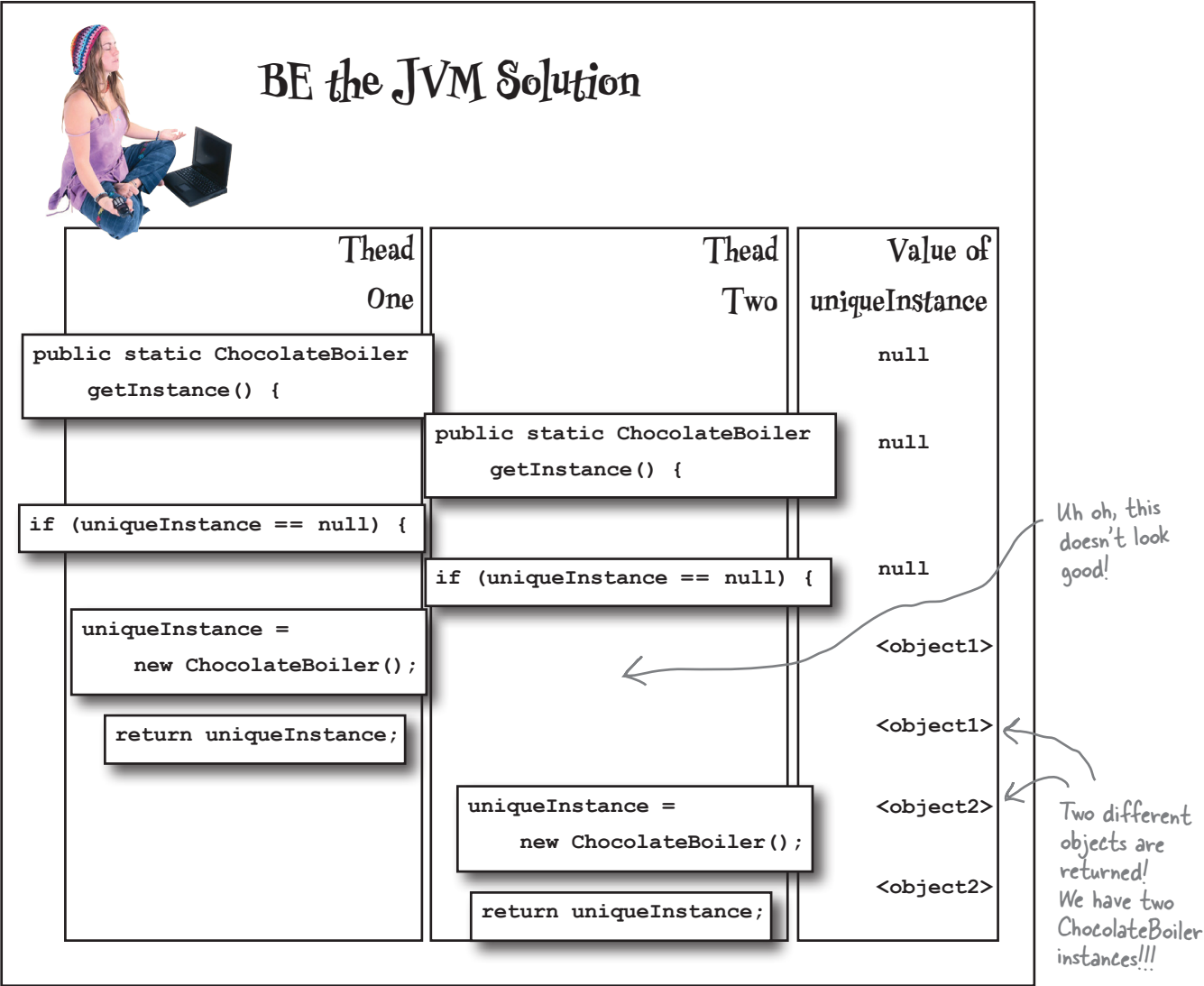


ACROSS

3. Company that produces boilers.
6. An incorrect implementation caused this to overflow.
7. The Singleton Pattern has one.
10. To totally defeat the new constructor, we have to declare the constructor _____.
12. The classic implementation doesn't handle this.
13. Singleton provides a single instance and _____ (three words).
14. An easy way to create Singletons in Java.
15. The Singleton was embarrassed it had no public _____.
16. A Singleton is a class that manages an instance of _____.

DOWN

1. Added to chocolate in the boiler.
2. Flawed multithreading approach if not using Java 5 or later (two words).
3. It was "one of a kind."
4. Multiple _____ can cause problems (two words).
5. If you don't need to worry about lazy instantiation, you can create your instance _____.
8. One advantage over global variables: _____ creation.
9. Chocolate capital of the USA.
11. Singleton ensures only one of these exists.





Sharpen your pencil Solution

Can you help Choc-O-Holic improve their ChocolateBoiler class by turning it into a Singleton?

```
public class ChocolateBoiler {
    private boolean empty;
    private boolean boiled;

    private static ChocolateBoiler uniqueInstance;

    private ChocolateBoiler() {
        empty = true;
        boiled = false;
    }

    public static ChocolateBoiler getInstance() {
        if (uniqueInstance == null) {
            uniqueInstance = new ChocolateBoiler();
        }
        return uniqueInstance;
    }

    public void fill() {
        if (isEmpty()) {
            empty = false;
            boiled = false;
            // fill the boiler with a milk/chocolate mixture
        }
    }
    // rest of ChocolateBoiler code...
}
```

Sharpen your pencil Solution

For each solution, describe its applicability to the problem of fixing the Chocolate Boiler code:

Synchronize the getInstance() method:

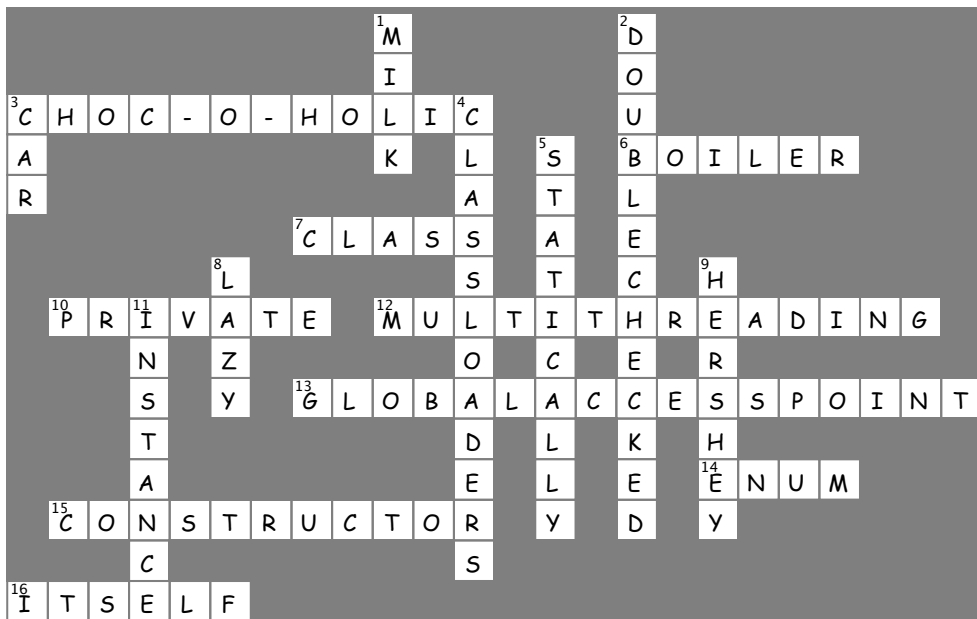
A straightforward technique that is guaranteed to work. We don't seem to have any performance concerns with the chocolate boiler, so this would be a good choice.

Use eager instantiation:

We are always going to instantiate the chocolate boiler in our code, so statically initializing the instance would cause no concerns. This solution would work as well as the synchronized method, although perhaps be less obvious to a developer familiar with the standard pattern.

Double-checked locking:

Given we have no performance concerns, double-checked locking seems like overkill. In addition, we'd have to ensure that we are running at least Java 5.

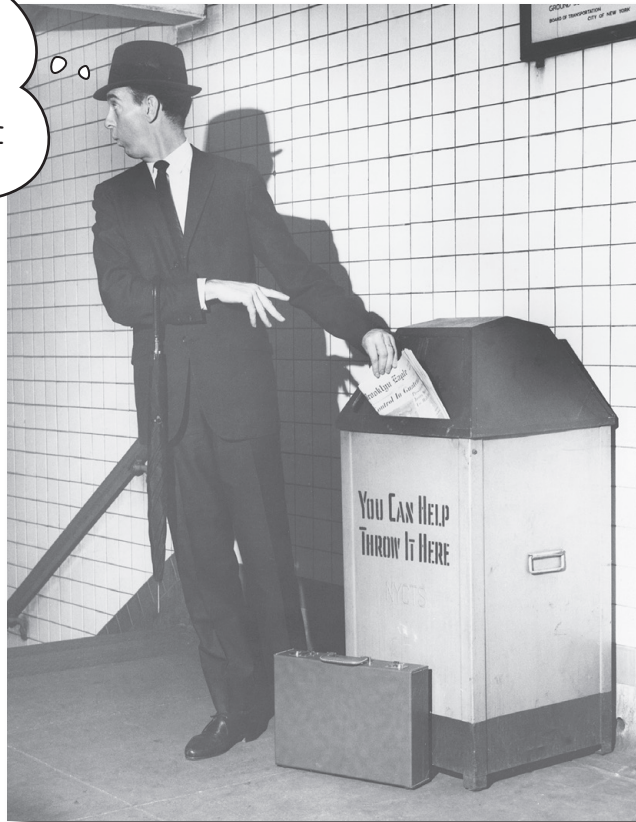


Design
Patterns
Crossword
Solution

6 the Command Pattern

✧ *Encapsulating Invocation* ✧

These top secret drop boxes have revolutionized the spy industry. I just drop in my request and people disappear, governments change overnight, and my dry cleaning gets done. I don't have to worry about when, where, or how; it just happens!



In this chapter, we take encapsulation to a whole new level: we're going to encapsulate method invocation. That's right—by encapsulating method invocation, we can crystallize pieces of computation so that the object invoking the computation doesn't need to worry about how to do things, it just uses our crystallized method to get it done. We can also do some wickedly smart things with these encapsulated method invocations, like save them away for logging or reuse them to implement undo functionality in our code.



Home Automation or Bust, Inc.
1221 Industrial Avenue, Suite 2000
Future City, IL 62914

Greetings!

I recently received a demo and briefing from Johnny Hurricane, CEO of Weather-O-Rama, on their new expandable weather station. I have to say, I was so impressed with the software architecture that I'd like to ask you to design the API for our new Home Automation Remote Control. In return for your services we'd be happy to handsomely reward you with stock options in Home Automation or Bust, Inc.

You should have already received a prototype of our ground-breaking remote control for your perusal. The remote control features seven programmable slots (each can be assigned to a different household device) along with corresponding on/off buttons for each. The remote also has a global undo button.

I'm also attaching to this email a set of Java classes that were created by various vendors to control home automation devices such as lights, fans, hot tubs, audio equipment, and other similar controllable appliances.

We'd like you to create an API for programming the remote so that each slot can be assigned to control a device or set of devices. Note that it is important that we be able to control all the current devices, and also any future devices that the vendors may supply.

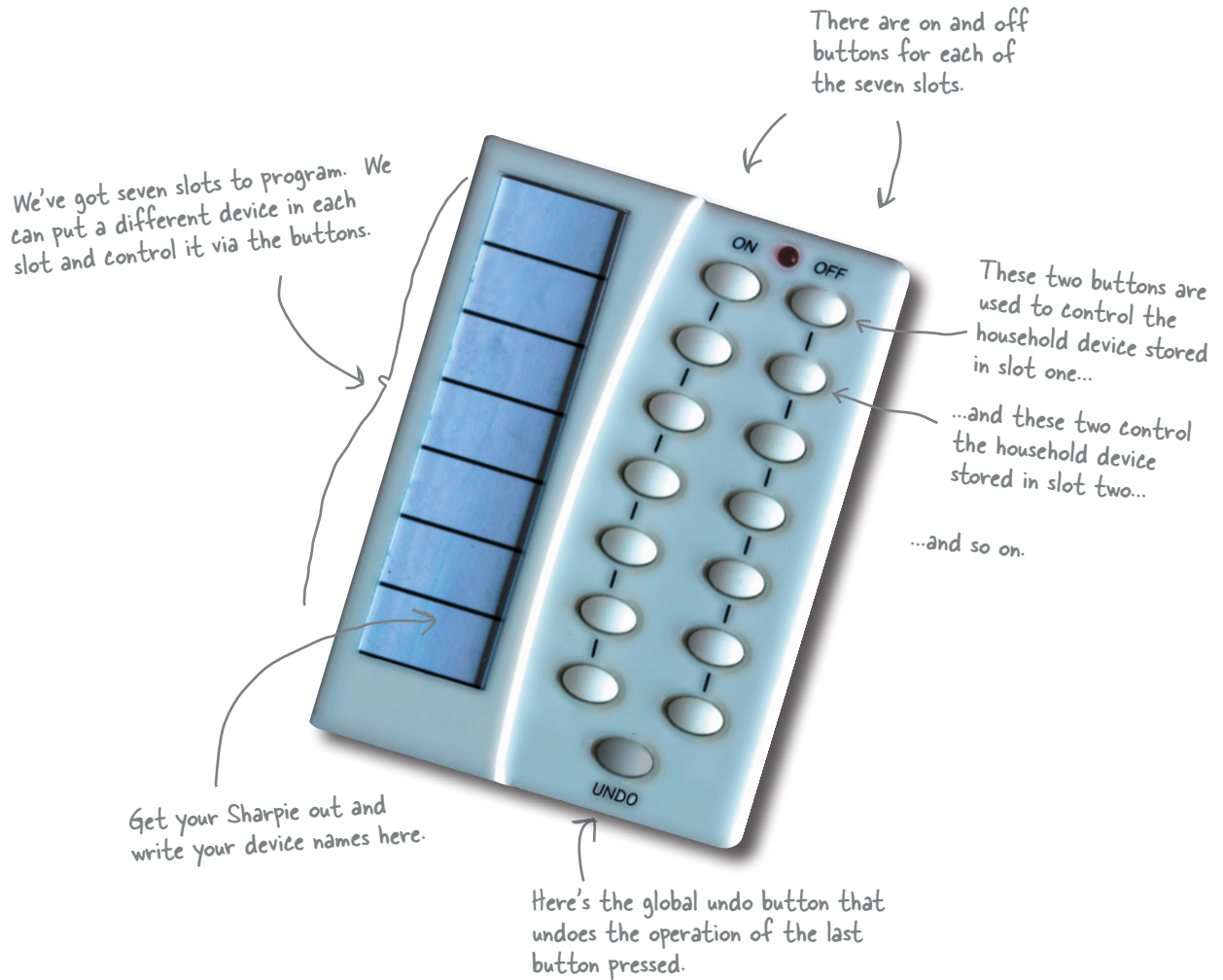
Given the work you did on the Weather-O-Rama weather station, we know you'll do a great job on our remote control!

We look forward to seeing your design.

Sincerely,

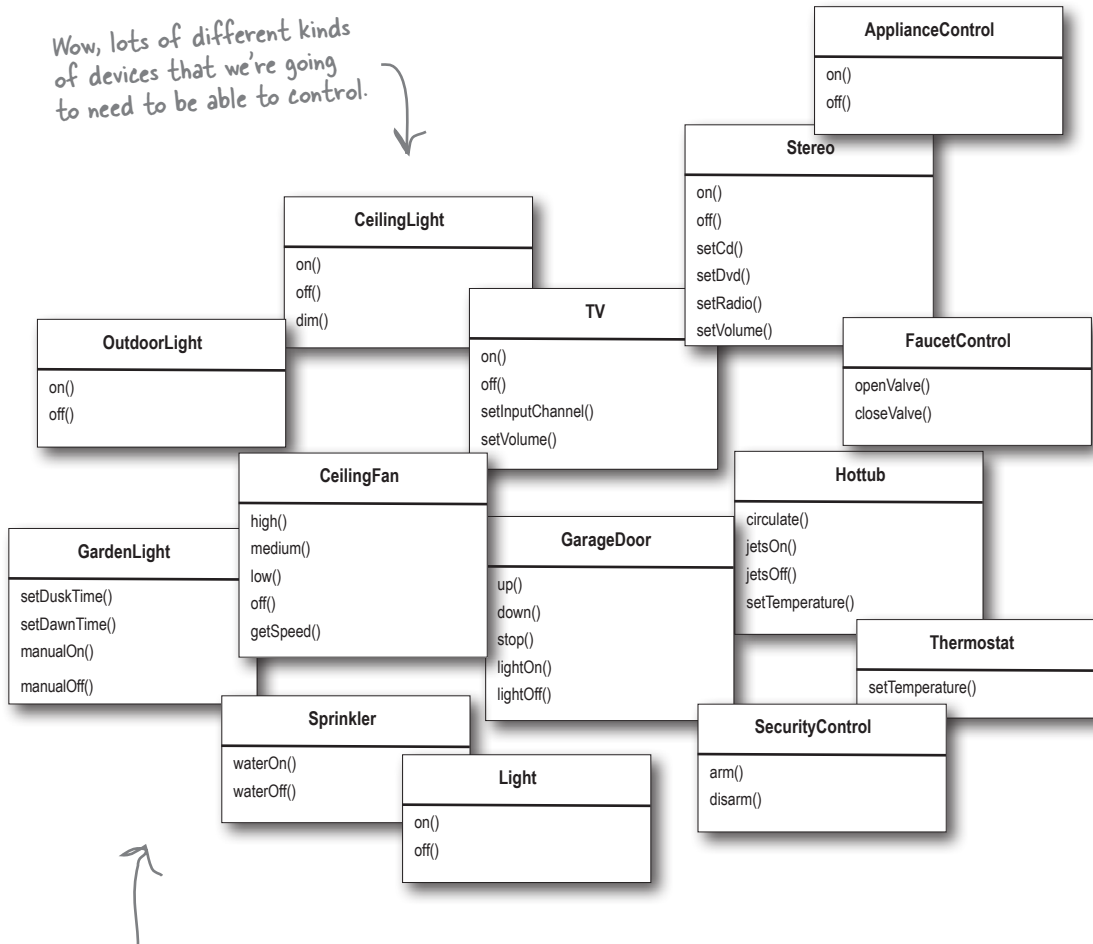
Bill Thompson, CEO

Free hardware! Let's check out the Remote Control...



Taking a look at the vendor classes

Let's check out the vendor classes the CEO attached to his email. These should give you some idea of the interfaces of the objects we need to control from the remote.



And some very different kinds of interfaces across these devices.

It looks like we have quite a set of classes here, and not a lot of industry effort to come up with a set of common interfaces. Not only that, it sounds like we can expect more of these classes in the future. Designing a remote control API is going to be interesting. Let's get on to the design.

Cubicle Conversation

Your teammates are already discussing how to design the remote control API...



Mary: Yes, I thought we'd see a bunch of classes with `on()` and `off()` methods, but here we've got methods like `dim()`, `setTemperature()`, `setVolume()`, and `setInputChannel()`, and `waterOn()`.

Sue: Not only that, it sounds like we can expect more vendor classes in the future with methods just as diverse.

Mary: I think it's important we view this as a separation of concerns.

Sue: Meaning?

Mary: What I mean is that the remote should know how to interpret button presses and make requests, but it shouldn't know a lot about home automation or how to turn on a hot tub.

Sue: But if the remote is dumb and just knows how to make generic requests, how do we design the remote so that it can invoke an action that, say, turns on a light or opens a garage door?

Mary: I'm not sure, but we don't want the remote to have to know the specifics of the vendor classes.

Sue: What do you mean?

Mary: We don't want the remote to consist of a set of if statements, like "if `slot1 == Light`, then `light.on()`, else if `slot1 == Hottub` then `hottub.jetsOn()`". We know that is a bad design.

Sue: I agree. Whenever a new vendor class comes out, we'd have to go in and modify the code, potentially creating bugs and more work for ourselves!



↑
Joe

Hey, I couldn't help overhearing. Since Chapter 1 I've been boning up on Design Patterns. There's a pattern called "Command Pattern" I think might help.

Mary: Yeah? Tell us more.

Joe: The Command Pattern allows you to decouple the requester of an action from the object that actually performs the action. So, here the requester would be the remote control and the object that performs the action would be an instance of one of your vendor classes.

Sue: How is that possible? How can we decouple them? After all, when I press a button, the remote has to turn on a light.

Joe: You can do that by introducing *command objects* into your design. A command object encapsulates a request to do something (like turn on a light) on a specific object (say, the living room light object). So, if we store a command object for each button, when the button is pressed we ask the command object to do some work. The remote doesn't have any idea what the work is, it just has a command object that knows how to talk to the right object to get the work done. So, you see, the remote is decoupled from the light object!

Sue: This certainly sounds like it's going in the right direction.

Mary: Still, I'm having a hard time wrapping my head around the pattern.

Joe: Given that the objects are so decoupled, it's a little difficult to picture how the pattern actually works.

Mary: Let me see if I at least have the right idea: using this pattern, we could create an API in which these command objects can be loaded into button slots, allowing the remote code to stay very simple. And the command objects encapsulate how to do a home automation task along with the object that needs to do it.

Joe: Yes, I think so. I also think this pattern can help you with that undo button, but I haven't studied that part yet.

Mary: This sounds really encouraging, but I think I have a bit of work to do to really "get" the pattern.

Sue: Me too.

Meanwhile, back at the Diner..., or, A brief introduction to the Command Pattern

As Joe said, it is a little hard to understand the Command Pattern by just hearing its description. But don't fear, we have some friends ready to help: remember our friendly diner from Chapter 1? It's been a while since we visited Alice, Flo, and the short-order cook, but we've got good reason for returning (beyond the food and great conversation): the diner is going to help us understand the Command Pattern.

So, let's take a short detour back to the diner and study the interactions between the customers, the waitress, the orders, and the short-order cook. Through these interactions, you're going to understand the objects involved in the Command Pattern and also get a feel for how the decoupling works. After that, we're going to knock out that remote control API.

Checking in at the Objectville Diner...

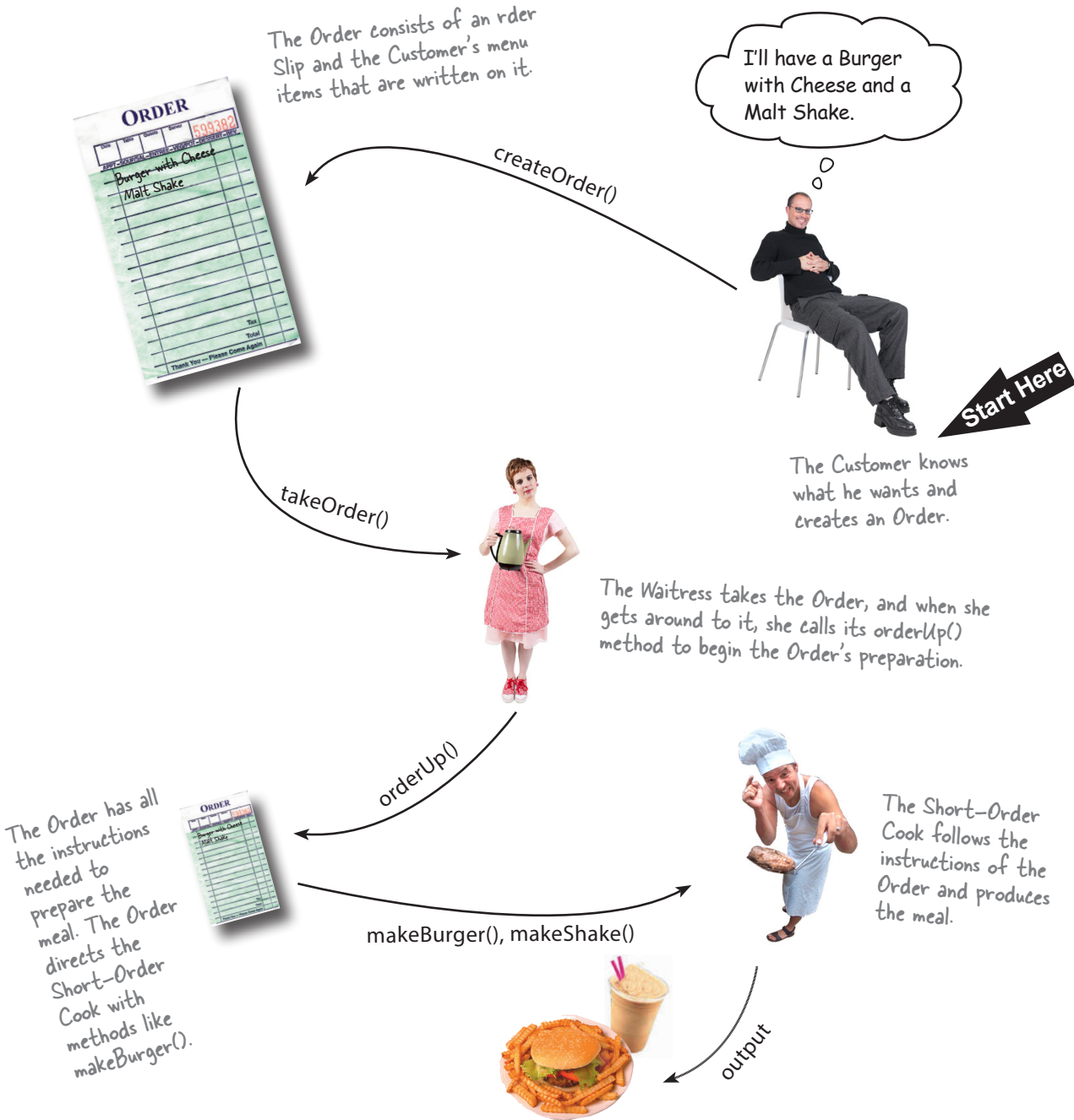


Okay, we all know how the Diner operates:



Let's study the interaction in a little more detail...

...and given this Diner is in Objectville, let's think about the object and method calls involved, too!



The Objectville Diner roles and responsibilities

An Order Slip encapsulates a request to prepare a meal.

Think of the Order Slip as an object that acts as a request to prepare a meal. Like any object, it can be passed around—from the Waitress to the order counter, or to the next Waitress taking over her shift. It has an interface that consists of only one method, `orderUp()`, that encapsulates the actions needed to prepare the meal. It also has a reference to the object that needs to prepare it (in our case, the Short-Order Cook). It's encapsulated in that the Waitress doesn't have to know what's in the Order or even who prepares the meal; she only needs to pass the slip through the order window and call "Order up!"



Okay, in real life a waitress would probably care what is on the order slip and who cooks it, but this is Objectville...work with us here!

The Waitress's job is to take Order Slips and invoke the `orderUp()` method on them.

The Waitress has it easy: take an Order from the Customer, continue helping customers until she makes it back to the order counter, and then invoke the `orderUp()` method to have the meal prepared. As we've already discussed, in Objectville, the Waitress really isn't worried about what's on the Order or who is going to prepare it; she just knows Order Slips have an `orderUp()` method she can call to get the job done.

Now, throughout the day, the Waitress's `takeOrder()` method gets parameterized with different Order Slips from different customers, but that doesn't faze her; she knows all Order Slips support the `orderUp()` method and she can call `orderUp()` any time she needs a meal prepared.

Don't ask me to cook, I just take orders and yell "Order up!"

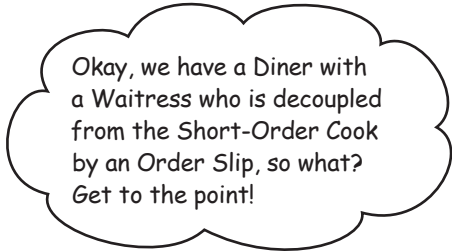


The Short-Order Cook has the knowledge required to prepare the meal.

The Short-Order Cook is the object that really knows how to prepare meals. Once the Waitress has invoked the `orderUp()` method; the Short-Order Cook takes over and implements all the methods that are needed to create meals. Notice the Waitress and the Cook are totally decoupled: the Waitress has Order Slips that encapsulate the details of the meal; she just calls a method on each Order to get it prepared. Likewise, the Cook gets his instructions from the Order Slip; he never needs to directly communicate with the Waitress.



You can definitely say the Waitress and I are decoupled. She's not even my type!



Patience, we're getting there...

Think of the Diner as a model for an OO design pattern that allows us to separate an object making a request from the objects that receive and execute those requests. For instance, in our remote control API, we need to separate the code that gets invoked when we press a button from the objects of the vendor-specific classes that carry out those requests. What if each slot of the remote held an object like the Diner's Order Slip object? Then, when a button is pressed, we could just call the equivalent of the `orderUp()` method on this object and have the lights turn on without the remote knowing the details of how to make those things happen or what objects are making them happen.

Now, let's switch gears a bit and map all this Diner talk to the Command Pattern...

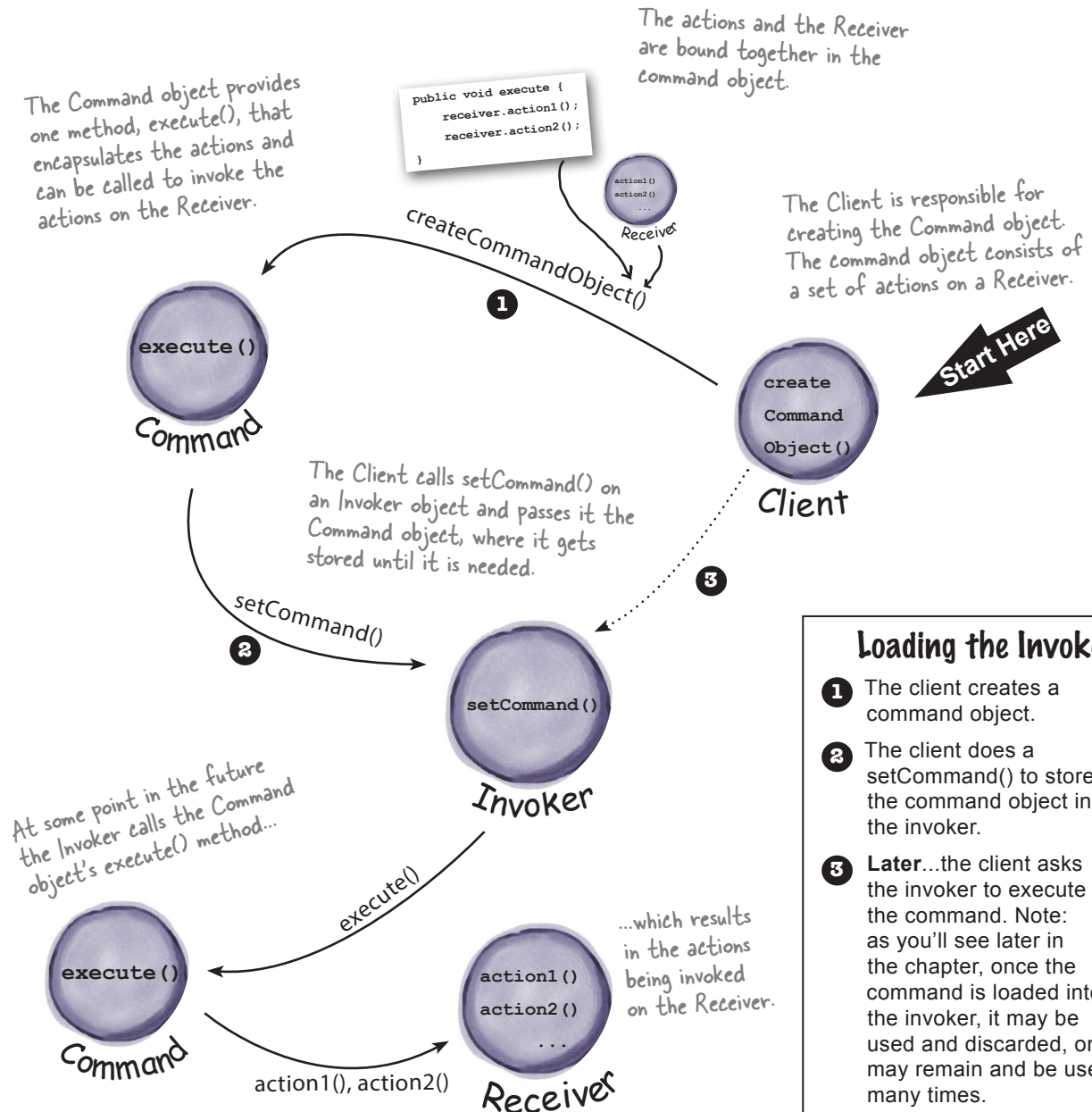


Before we move on, spend some time studying the diagram two pages back along with Diner roles and responsibilities until you think you've got a handle on the Objectville Diner objects and relationships. Once you've done that, get ready to nail the Command Pattern!



From the Diner to the Command Pattern

Okay, we've spent enough time in the Objectville Diner that we know all the personalities and their responsibilities quite well. Now we're going to rework the Diner diagram to reflect the Command Pattern. You'll see that all the players are the same; only the names have changed.



Loading the Invoker

- 1** The client creates a command object.
- 2** The client does a `setCommand()` to store the command object in the invoker.
- 3** Later...the client asks the invoker to execute the command. Note: as you'll see later in the chapter, once the command is loaded into the invoker, it may be used and discarded, or it may remain and be used many times.

WHO DOES WHAT?

Match the diner objects and methods with the corresponding names from the Command Pattern.

Diner	Command Pattern
Waitress	Command
Short-Order Cook	execute()
orderUp()	Client
Order	Invoker
Customer	Receiver
takeOrder()	setCommand()